

API REST con PHP

DESARROLLO WEB AVANZADO

DR. JOSE ALFONSO AGUILAR CALDERÓN

CONTENIDO



EXPLICACIÓN DETALLADA DEL
PROYECTO RESTFUL



CRUD DE PRODUCTOS CON
ARQUITECTURA PROFESIONAL

Objetivo de la API



PERMITIR GESTIONAR
PRODUCTOS MEDIANTE
HTTP



OPERACIONES: CREAR,
LEER, ACTUALIZAR Y
ELIMINAR



USO EN APLICACIONES
WEB Y MÓVILES

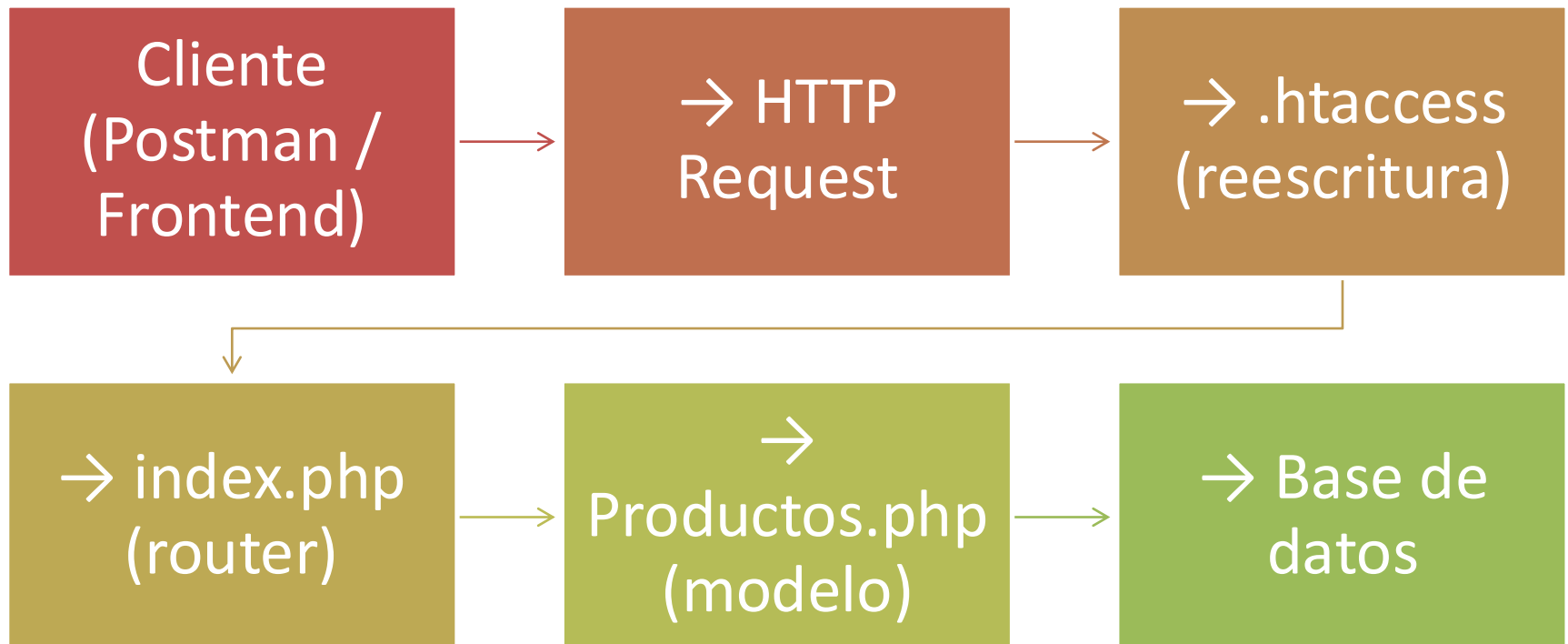
¿Qué es esta API?

Es una API
REST

Expone un
recurso:
productos

Se accede
mediante
endpoints

Arquitectura del sistema



Estructura del proyecto

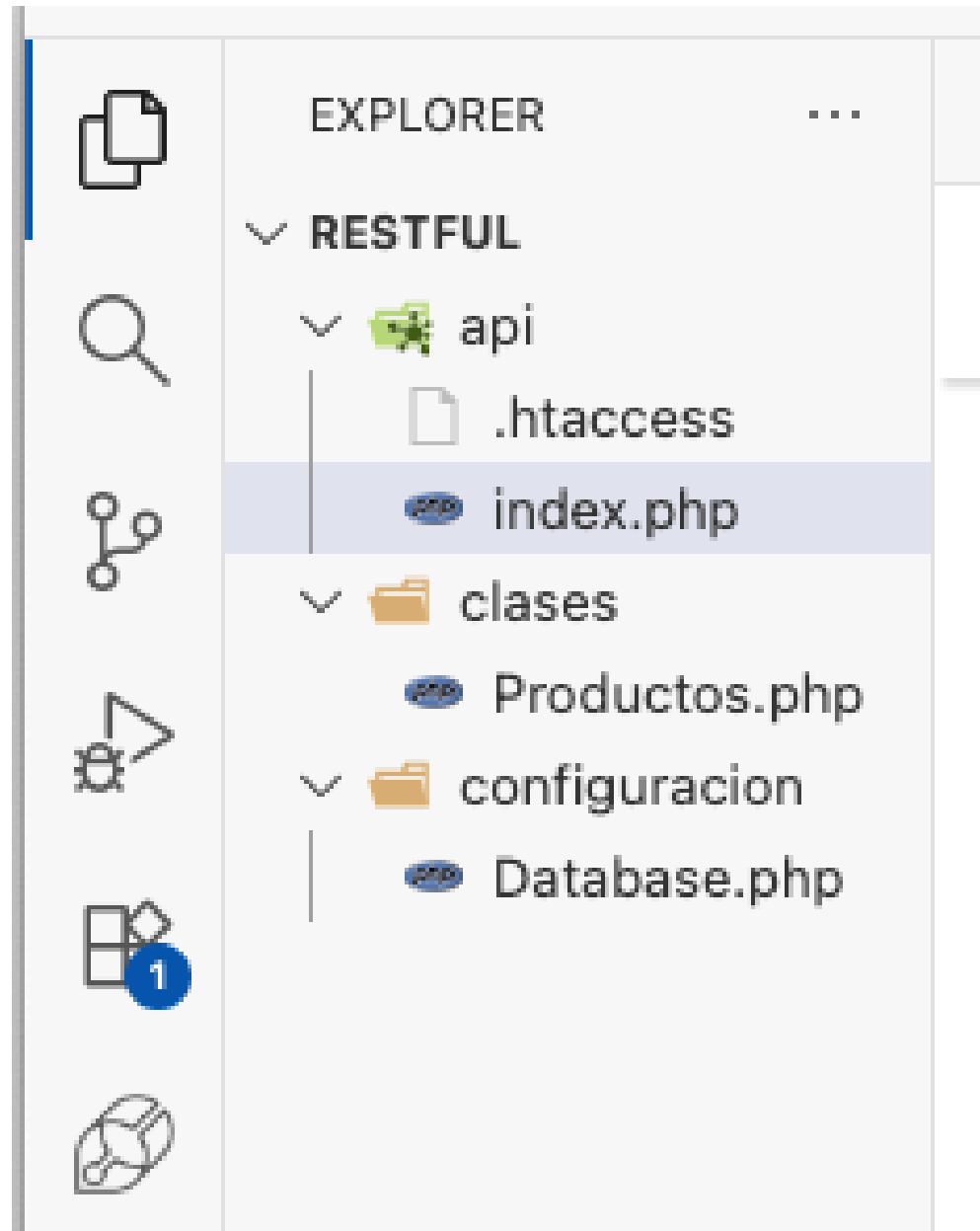
configuracion/Database.php → conexión BD

clases/Productos.php → lógica negocio

api/index.php → router

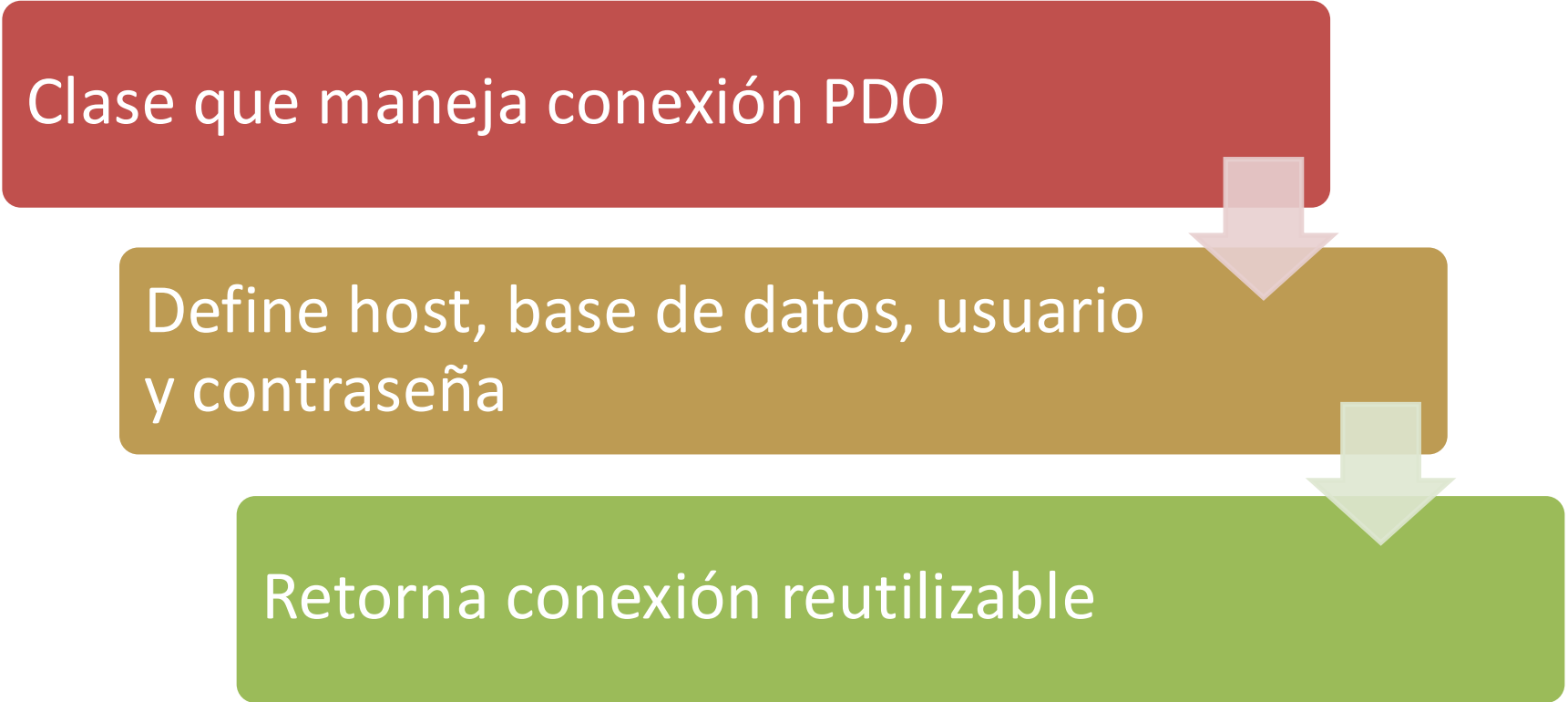
api/.htaccess → URLs limpias

Mi editor
de código
fuente
quedaría
así:



Database.php

Clase que maneja conexión PDO



```
graph TD; A[Clase que maneja conexión PDO] --> B[Define host, base de datos, usuario y contraseña]; B --> C[Retorna conexión reutilizable];
```

Define host, base de datos, usuario y contraseña

Retorna conexión reutilizable

TABLA PRODUCTOS

- CREATE DATABASE IF NOT EXISTS productos;
USE productos;

```
CREATE TABLE IF NOT EXISTS productos (  
idProducto INT AUTO_INCREMENT PRIMARY KEY,  
nombreproducto VARCHAR(100) NOT NULL,  
descripcion TEXT NOT NULL,  
precioCompra DECIMAL(10,2) NOT NULL,  
precioVenta DECIMAL(10,2) NOT NULL,  
existencia INT NOT NULL  
);
```

Database.php

configuracion > Database.php

```
1  <?php
2  class Database {
3      private $host = "localhost";
4      private $db_name = "productos";
5      private $username = "root";
6      private $password = "";
7      public $conn;
8
9      public function getConnection() {
10         $this->conn = null;
11
12         try {
13             $this->conn = new PDO(
14                 "mysql:host=" . $this->host . ";dbname=" . $this->db_name,
15                 $this->username,
16                 $this->password
17             );
18             $this->conn->exec("set names utf8");
19         } catch(PDOException $exception) {
20             echo "Error en conexión a la base de datos: " . $exception->getMessage();
21         }
22
23         return $this->conn;
24     }
25 }
26 ?>
```

Productos.php

Clase modelo

Contiene propiedades del producto

Métodos:

- getProductos() (devuelve todos)

- getProducto() (devuelve 1)

- setProductos() (guarda 1)

- updateProducto() (actualiza 1)

- borrarProducto() (elimina 1)

Productos.php (1)

clases > Productos.php

```
1  <?php
2  class Productos {
3
4      // Conexión
5      private $conn;
6
7      // Tabla de la BD a utilizar
8      private $tabla = "productos";
9
10     // Columnas de la tabla
11     public $idProducto;
12     public $nombreproducto;
13     public $descripcion;
14     public $precioCompra;
15     public $precioVenta;
16     public $existencia;
17
18     // Establecer conexión con la BD
19     public function __construct($db) {
20         $this->conn = $db;
21     }
22
```

Productos.php (2)

```
23 // Obtener todos los productos
24 public function getProductos() {
25     $consultaSQL = "SELECT idProducto, nombre, descripcion, precioCompra, precioVenta,
26         existencia
27         FROM " . $this->tabla . " ";
28
29     // En las diapositivas aparece 'nombre', pero la tabla maneja 'nombreproducto'.
30     // Para que funcione con la tabla propuesta, dejamos así:
31     $consultaSQL = "SELECT idProducto, nombreproducto, descripcion, precioCompra,
32         precioVenta, existencia
33         FROM " . $this->tabla;
34
35     $stmt = $this->conn->prepare($consultaSQL);
36     $stmt->execute();
37
38     return $stmt;
39 }
40
```

Productos.php (3)

```
41 // Devuelve un producto buscado por idProducto
42 public function getProducto() {
43     $consultaSQL = "SELECT
44         idProducto,
45         nombreproducto,
46         descripcion,
47         precioCompra,
48         precioVenta,
49         existencia
50     FROM
51         " . $this->tabla . "
52     WHERE
53         idProducto = ?
54     LIMIT 0,1";
55
56     $stmt = $this->conn->prepare($consultaSQL);
57     $stmt->bindParam(1, $this->idProducto);
58     $stmt->execute();
59
60     $dataRow = $stmt->fetch(PDO::FETCH_ASSOC);
61
62     if ($dataRow) {
63         $this->nombreproducto = $dataRow['nombreproducto'];
64         $this->descripcion = $dataRow['descripcion'];
65         $this->precioCompra = $dataRow['precioCompra'];
66         $this->precioVenta = $dataRow['precioVenta'];
67         $this->existencia = $dataRow['existencia'];
68         return true;
69     }
70
71     return false;
72 }
```

Productos.php (4)

```
74 // Insertar un producto
75 public function setProductos() {
76     $consultaSQL = "INSERT INTO
77         |         |         |         |         |
78         |         |         |         |         | SET
79         |         |         |         |         | nombreproducto = :nombreproducto,
80         |         |         |         |         | descripcion = :descripcion,
81         |         |         |         |         | precioCompra = :precioCompra,
82         |         |         |         |         | precioVenta = :precioVenta,
83         |         |         |         |         | existencia = :existencia";
84
85     $stmt = $this->conn->prepare($consultaSQL);
86
87     // Limpiar caracteres especiales
88     $this->nombreproducto = htmlspecialchars(strip_tags($this->nombreproducto));
89     $this->descripcion = htmlspecialchars(strip_tags($this->descripcion));
90     $this->precioCompra = htmlspecialchars(strip_tags($this->precioCompra));
91     $this->precioVenta = htmlspecialchars(strip_tags($this->precioVenta));
92     $this->existencia = htmlspecialchars(strip_tags($this->existencia));
93
94     // Enlazar datos
95     $stmt->bindParam(":nombreproducto", $this->nombreproducto);
96     $stmt->bindParam(":descripcion", $this->descripcion);
97     $stmt->bindParam(":precioCompra", $this->precioCompra);
98     $stmt->bindParam(":precioVenta", $this->precioVenta);
99     $stmt->bindParam(":existencia", $this->existencia);
100
101     if ($stmt->execute()) {
102         |     return true;
103     }
104
105     return false;
106 }
```

Productos.php (5)

```
107
108 // Borrar producto
109 public function borrarProducto() {
110     $consultaSQL = "DELETE FROM " . $this->tabla . " WHERE idProducto = ?";
111     $stmt = $this->conn->prepare($consultaSQL);
112
113     $this->idProducto = htmlspecialchars(strip_tags($this->idProducto));
114     $stmt->bindParam(1, $this->idProducto);
115
116     if ($stmt->execute()) {
117         return true;
118     }
119
120     return false;
121 }
122
```

Productos.php

(6)

```
123     public function updateProducto() {
124         $consultaSQL = "UPDATE " . $this->tabla . "
125         SET
126             nombreproducto = :nombreproducto,
127             descripcion = :descripcion,
128             precioCompra = :precioCompra,
129             precioVenta = :precioVenta,
130             existencia = :existencia
131         WHERE
132             idProducto = :idProducto";
133
134         $stmt = $this->conn->prepare($consultaSQL);
135
136         $this->idProducto = htmlspecialchars(strip_tags($this->idProducto));
137         $this->nombreproducto = htmlspecialchars(strip_tags($this->nombreproducto));
138         $this->descripcion = htmlspecialchars(strip_tags($this->descripcion));
139         $this->precioCompra = htmlspecialchars(strip_tags($this->precioCompra));
140         $this->precioVenta = htmlspecialchars(strip_tags($this->precioVenta));
141         $this->existencia = htmlspecialchars(strip_tags($this->existencia));
142
143         $stmt->bindParam(':idProducto', $this->idProducto);
144         $stmt->bindParam(':nombreproducto', $this->nombreproducto);
145         $stmt->bindParam(':descripcion', $this->descripcion);
146         $stmt->bindParam(':precioCompra', $this->precioCompra);
147         $stmt->bindParam(':precioVenta', $this->precioVenta);
148         $stmt->bindParam(':existencia', $this->existencia);
149
150         return $stmt->execute();
151     }
152 }
153 ?>
```

index.php
(Router)

Recibe todas las
peticiones

Detecta método HTTP

Analiza la URL

Decide qué operación
ejecutar

Index.php(1)

```
api > index.php
1  <?php
2  header("Content-Type: application/json; charset=UTF-8");
3  header("Access-Control-Allow-Origin: *");
4  header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");
5  header("Access-Control-Allow-Headers: Content-Type, Authorization, X-Requested-With");
6
7  if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
8      http_response_code(200);
9      exit;
10 }
11
12 require_once "../configuracion/database.php";
13 require_once "../clases/Productos.php";
14
15 $database = new Database();
16 $db = $database->getConnection();
17 $producto = new Productos($db);
18
19 // Método HTTP
20 $method = $_SERVER['REQUEST_METHOD'];
21
22 // Ruta solicitada
23 $uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
24
25 // Quitar la base del proyecto
26 $basePath = '/RESTful/api';
27 $endpoint = str_replace($basePath, '', $uri);
28 $endpoint = trim($endpoint, '/');
29
30 // Separar segmentos
31 $segments = explode('/', $endpoint);
32
33 // Ejemplos:
34 // /productos      => ["productos"]
35 // /productos/1    => ["productos", "1"]
36
```

Index.php(2)

```
36
37     if ($segments[0] !== 'productos') {
38         http_response_code(404);
39         echo json_encode(["message" => "Recurso no encontrado"]);
40         exit;
41     }
42
43     // GET /productos
44     if ($method === 'GET' && count($segments) === 1) {
45         $stmt = $producto->getProductos();
46         $total = $stmt->rowCount();
47
48         if ($total > 0) {
49             $productos = [];
50
51             while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
52                 $productos[] = $row;
53             }
54
55             http_response_code(200);
56             echo json_encode($productos);
57         } else {
58             http_response_code(404);
59             echo json_encode(["message" => "No se encontraron productos"]);
60         }
61         exit;
62     }
63
```

Index.php(3)

```
64 // GET /productos/1
65 if ($method === 'GET' && count($segments) === 2 && is_numeric($segments[1])) {
66     $producto->idProducto = (int)$segments[1];
67
68     if ($producto->getProducto()) {
69         http_response_code(200);
70         echo json_encode([
71             "idProducto" => $producto->idProducto,
72             "nombreproducto" => $producto->nombreproducto,
73             "descripcion" => $producto->descripcion,
74             "precioCompra" => $producto->precioCompra,
75             "precioVenta" => $producto->precioVenta,
76             "existencia" => $producto->existencia
77         ]);
78     } else {
79         http_response_code(404);
80         echo json_encode(["message" => "Producto no encontrado"]);
81     }
82     exit;
83 }
84
85 // POST /productos
86 if ($method === 'POST' && count($segments) === 1) {
87
88     $data = json_decode(file_get_contents("php://input"));
89
90     // VALIDACIONES
91     $errores = [];
92
93     if (empty($data->nombreproducto)) {
94         $errores[] = "El nombre del producto es obligatorio";
95     }
96
97     if (!isset($data->precioCompra) || $data->precioCompra < 0) {
98         $errores[] = "El precio de compra no puede ser negativo";
99     }
```

Index.php(4)

```
100
101     if (!isset($data->precioVenta) || $data->precioVenta < 0) {
102         $errores[] = "El precio de venta no puede ser negativo";
103     }
104
105     if (!isset($data->existencia) || $data->existencia < 0) {
106         $errores[] = "La existencia no puede ser negativa";
107     }
108
109     // VALIDACIÓN PRO ADICIONAL: El precio de venta no puede ser menor al de compra
110     if (isset($data->precioVenta, $data->precioCompra) &&
111         $data->precioVenta < $data->precioCompra) {
112         $errores[] = "El precio de venta no puede ser menor al de compra";
113     }
114
115     // SI HAY ERRORES
116     if (!empty($errores)) {
117         http_response_code(400);
118         echo json_encode([
119             "status" => "error",
120             "errores" => $errores
121         ]);
122         exit;
123     }
124
125     // SI TODO OK
126     $producto->nombreproducto = $data->nombreproducto;
127     $producto->descripcion = $data->descripcion ?? '';
128     $producto->precioCompra = $data->precioCompra;
129     $producto->precioVenta = $data->precioVenta;
130     $producto->existencia = $data->existencia;
131
```

Index.php(5)

```
132         if ($producto->setProductos()) {
133             http_response_code(201);
134             echo json_encode([
135                 "status" => "success",
136                 "message" => "Producto creado correctamente"
137             ]);
138         } else {
139             http_response_code(500);
140             echo json_encode([
141                 "status" => "error",
142                 "message" => "Error al guardar"
143             ]);
144         }
145
146         exit;
147     }
148
149     // PUT /productos/1
150     if ($method === 'PUT' && count($segments) === 2) {
151
152         $data = json_decode(file_get_contents("php://input"));
153         $producto->idProducto = (int)$segments[1];
154
155         $errores = [];
156
157         if (empty($data->nombrepProducto)) {
158             $errores[] = "El nombre es obligatorio";
159         }
160
161         if ($data->precioCompra < 0) {
162             $errores[] = "Precio de compra inválido";
163         }
164     }
```

Index.php(6)

```
165     if ($data->precioVenta < 0) {
166         $errores[] = "Precio de venta inválido";
167     }
168
169     if ($data->existencia < 0) {
170         $errores[] = "Existencia inválida";
171     }
172
173     if ($data->precioVenta < $data->precioCompra) {
174         $errores[] = "El precio de venta no puede ser menor al de compra";
175     }
176
177     if (!empty($errores)) {
178         http_response_code(400);
179         echo json_encode([
180             "status" => "error",
181             "errores" => $errores
182         ]);
183         exit;
184     }
185
186     $producto->nombreproducto = $data->nombreproducto;
187     $producto->descripcion = $data->descripcion;
188     $producto->precioCompra = $data->precioCompra;
189     $producto->precioVenta = $data->precioVenta;
190     $producto->existencia = $data->existencia;
191
192     if ($producto->updateProducto()) {
193         echo json_encode(["status" => "success", "message" => "Actualizado"]);
194     } else {
195         echo json_encode(["status" => "error", "message" => "Error al actualizar"]);
196     }
197
198     exit;
199 }
```

Index.php(7)

```
200
201 // DELETE /productos/1
202 if ($method === 'DELETE' && count($segments) === 2 && is_numeric($segments[1])) {
203     $producto->idProducto = (int)$segments[1];
204
205     if ($producto->borrarProducto()) {
206         http_response_code(200);
207         echo json_encode(["message" => "Producto eliminado"]);
208     } else {
209         http_response_code(500);
210         echo json_encode(["message" => "No se pudo eliminar"]);
211     }
212     exit;
213 }
214
215 http_response_code(405);
216 echo json_encode(["message" => "Método no permitido o ruta inválida"]);
```

Crear mi .htaccess

- Un archivo **.htaccess** (HyperText Access) es un archivo de configuración utilizado por el servidor web **Apache HTTP Server** que permite **modificar el comportamiento del servidor a nivel de carpeta (directorio)** sin necesidad de cambiar la configuración global.
- Se usa principalmente para controlar cómo funciona tu sitio web en aspectos como:
- **1. URLs amigables (Rewrite)**
 - Permite transformar URLs complejas en rutas limpias (muy usado en APIs REST y MVC).
- **2. Redirecciones**
 - Puedes redirigir páginas o dominios.
- **3. Seguridad**
 - Bloquear acceso a archivos o carpetas, restringir por IP
- **4. Configuración de errores**
 - Personalizar páginas de error.

Creatr .htaccess

```
api > .htaccess
1 RewriteEngine On
2 RewriteBase /RESTful/api/
3
4 RewriteCond %{REQUEST_FILENAME} !-f
5 RewriteCond %{REQUEST_FILENAME} !-d
6 RewriteRule ^ index.php [QSA,L]
```

Diseño REST de endpoints

GET /productos

GET /productos/{id}

POST /productos

PUT /productos/{id}

DELETE /productos/{id}

Flujo de una petición

Cliente envía request

Router analiza ruta

Se llama al método
correspondiente

Se consulta la BD

Se devuelve JSON

Uso en Postman

Seleccionar método HTTP



Ingresar URL



Configurar Body JSON



Enviar petición

Ejemplo
POST

Crear producto
Harley

Body con nombre,
precio, existencia

Ejemplo POST(2)

The screenshot displays a REST client interface for a POST request. The top bar shows the request method as **POST** and the URL as `http://localhost/RESTful/api/productos`. Below the URL bar, tabs for **Docs**, **Params**, **Authorization**, **Headers**, **Body**, **Scripts**, **Tests**, and **Settings** are visible. The **Body** tab is selected, showing a JSON payload in a code editor. The payload is a JSON object with the following fields: `"nombreproducto": "Harley-Davidson Road King 1450cc"`, `"descripcion": "Motocicleta Harley-Davidson modelo Electra Glide, motor Twin Cam."`, `"precioCompra": 280000`, `"precioVenta": 325000`, and `"existencia": 3`. Below the code editor, the response section shows a status of **201 Created** with a response time of 16 ms and a size of 529 B. The response body is also in JSON format, containing `"status": "success"` and `"message": "Producto creado correctamente"`.

Ejemplo REST-Clase > **POST Producto** Save Share

POST `http://localhost/RESTful/api/productos` Send

Docs **Params** **Authorization** **Headers** 9 **Body** **Scripts** **Tests** **Settings** **Cookies**

raw **JSON** Schema Beautify

```
1 {
2   "nombreproducto": "Harley-Davidson Road King 1450cc",
3   "descripcion": "Motocicleta Harley-Davidson modelo Electra Glide, motor
4     Twin Cam.",
5   "precioCompra": 280000,
6   "precioVenta": 325000,
7   "existencia": 3
8 }
```

Body 201 Created • 16 ms • 529 B • Save Response

{} **JSON** Preview Visualize

```
1 {
2   "status": "success",
3   "message": "Producto creado correctamente"
4 }
```

Validaciones implementadas

Nombre obligatorio

No precios negativos

No existencia negativa

Precio venta $\geq$ precio compra

Manejo de errores

- **Respuestas HTTP:**
 - 200 OK
 - 201 Created
 - 400 Bad Request
 - 404 Not Found
 - 500 Error servidor

GET productos

HTTP Ejemplo REST-Clase > GET Todos Save Share

GET Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body ↺ 200 OK · 13 ms · 1.15 KB · 🌐 Save Response ...

{} JSON ▶ Preview 🖨 Visualize ⌵

```

6         sistema de frenos ABS avanzado.",
7         "precioCompra": "285000.00",
8         "precioVenta": "335000.00",
9         "existencia": "5"
10      },
11      {
12        "idProducto": "2",
13        "nombreproducto": "Harley-Davidson Electra Glide 1450cc",
14        "descripcion": "Motocicleta Harley-Davidson modelo Electra Glide,
15                      motor Twin Cam.",
16        "precioCompra": "280000.00",
17        "precioVenta": "325000.00",
18        "existencia": "3"
19      }
20    ]
  
```

GET 1 Producto

HTTP Ejemplo REST-Clase > GET 1 Save Share

GET http://localhost/RESTful/api/productos/1 Send

Docs **Params** Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Refresh 200 OK • 17 ms • 723 B • Save Response ...

{ } JSON Preview Visualize ...

```
1 {  
2   "idProducto": "1",  
3   "nombreproducto": "Harley-Davidson Sportster S 1250 (Actualizada)",  
4   "descripcion": "Versión actualizada con mejoras en suspensión y sistema  
   de frenos ABS avanzado.",  
5   "precioCompra": "285000.00",  
6   "precioVenta": "335000.00",  
7   "existencia": "5"  
8 }
```

PUT 1 Producto

HTTP Ejemplo REST-Clase > PUT Producto Save Share

PUT http://localhost/RESTful/api/productos/1 Send

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "nombreproducto": "Harley-Davidson Sportster S 1250 (Actualizada)",
3   "descripcion": "Versión actualizada con mejoras en suspensión y sistema
4     de frenos ABS avanzado.",
5   "precioCompra": 285000,
6   "precioVenta": 335000,
7   "existencia": 5
8 }
```

Body 200 OK • 21 ms • 496 B • Save Response

JSON Preview Visualize

```
1 {
2   "message": "Producto actualizado"
3 }
```

DELETE 1 Producto

HTTP Ejemplo REST-Clase > DELETE Producto Save Share

DELETE ▼ Send ▼

Docs **Params** Authorization Headers 6 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body ▼ 🕒 200 OK • 25 ms • 494 B • 🌐 | e.g. Save Response ...

{} JSON ▼ ▶ Preview 🖼️ Visualize ▼ 🔍 🔑 📄 🔗

```
1 {  
2   "message": "Producto eliminado"  
3 }
```

¿Una API o varias?

- Es una sola API
- Contiene múltiples endpoints
- Cada endpoint representa una operación

Estructura de mi Proyecto

No se está obligado a usar **MVC** para que sea una **API REST**.

REST define **cómo expones los recursos y endpoints**, no te obliga a una arquitectura interna específica.

Lo importante es que nuestra API tenga:

- un **punto de entrada** (index.php)
- manejo de **rutasy endpoints**
- clases o archivos que procesen la lógica
- conexión a base de datos separada
- respuestas en **JSON**
- uso correcto de métodos HTTP: GET, POST, PUT, DELETE

Directorios

api/

- Aquí está la parte pública de la API.
- .htaccess redirige las peticiones.
- index.php recibe la solicitud y decide qué hacer.

clases/

- Aquí está la lógica de acceso a datos o procesamiento (CRUD).

configuracion/

- Aquí está la configuración de conexión.

Conclusión

- Ejemplo de API REST profesional
- Escalable. Lista para integrarse con frontend o apps móviles.
- Esta implementación no sigue estrictamente el patrón MVC, sino una estructura ligera para una API REST en PHP. La arquitectura se organiza en tres partes básicas:
 - un punto de entrada para recibir solicitudes HTTP,
 - clases para encapsular la lógica de acceso a datos,
 - un módulo de configuración para la conexión a la base de datos.
 - Aunque no utiliza controladores ni vistas, sigue siendo una API REST válida porque expone recursos mediante endpoints, utiliza métodos HTTP y devuelve respuestas en formato JSON.
- Estructura en inglés, como debe de ser: api (index y htacces), Models(Product.php) y Config(Database.php).